Avni Sanghvi

DSC 450 | Database Processing for Large-Scale Analytics| Spring 2026 | Assignment 2

Part 1

1. Using your logical schema from previous assignment (Q2), write the necessary SQL DDL script to create the tables. Be sure to specify every primary key and every foreign key. Make reasonable assumptions regarding the attribute domains (for example, setting every column to *VARCHAR2(100)* is not reasonable).

2. Write SQL INSERT statements to populate your database from Q2 with at least 3 customers, 2 restaurants, 4 orders, and 6 order items.

3. Using logical schema from previous assignment (Q4) write the necessary SQL DDL script to create the tables. Be sure to specify every primary key and every foreign key.

4. Write a python function to validate SQL insert statements. The function will take a string containing a SQL insert statement and print two kinds of messages. 1) "Invalid insert" or 2) Inserting [list of values] into [the target] table. The values and the table name would be based on each particular statement. For validating the statement, you only have to check that the insert statement starts with INSERT INTO and that statements ends with a semicolon (;).

Examples:

validateInsert("INSERT INTO Students VALUES (1, 'Jane', 'A-');")

- Inserting (1, 'Jane', 'A-') into Students table

validateInsert("INSERT INTO Students VALUES (1, 'Jane', 'A-')")

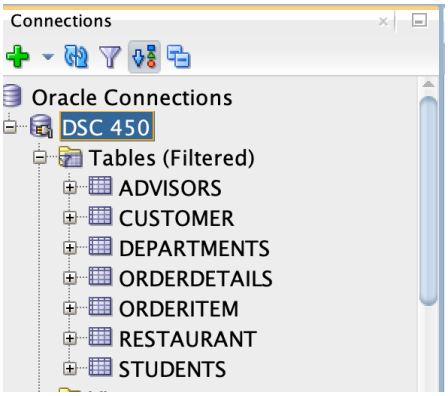
- Invalid insert

validateInsert("INSERT Students VALUES (1, 'Jane', B+);")

- Invalid insert

validateInsert("INSERT INTO Phones VALUES (42, '312-676-1213');")

- Inserting (42, '312-667-1213') into Phones table

→ 

→ Customer table:

	CUSTOMER_ID	NAME	EMAIL
1	101	Avni Sanghvi	asanghvi@gmail.com
2	102	Tanvi Sanghvi	tsanghvi@gmail.com
3	103	Chirag Khandhar	ckhandhar@gmail.com

→ Restaurant Table

	RESTAURANT_ID	NAME	ISOPEN
1	1	Restaurant1	Y
2	2	Restaurant2	Y

→ OrderDetails

	ORDER_ID	CUSTOMER_ID	RESTAURANT_ID	TOTAL_PRICE	STATUS
1	100	102	1	50	Delivered
2	104	101	2	35	Preparing
3	105	103	2	48	Received
4	103	101	1	80	Delivered
5	101	102	1	16	Preparing
6	106	103	2	25	Delivered

→ OrderItems

	ORDER_ID	ITEM_NAME	QUANTITY	PRICE
1	100	Item A	2	25
2	104	Item B	5	7
3	105	Item C	4	12
4	103	Item D	4	20
5	101	Item E	2	8
6	106	Item F	2	12.5

→ Coding Solution is in the .sql and .py file submitted.

Part 2

Consider a MEETING table that records information about meetings between clients and representatives in the company. Each record contains the names of the client and the representative's name as well as the office number, floor and the building. Finally, each record contains the city that the building is in and the date of the meeting. The table is in First Normal Form and the primary key is (Client, Office).
(Date, Client, Office, Floor, Building, City, Representative)

You are given the following functional dependencies:
Building → City
Office → Floor, Building, City
Client → Representative
Client, Office → Date

a. For the functional dependency Building → City, explain the redundancy problem and possible consequences through an example (you can make up your own building names as you see fit).

→ To understand the redundancy problem, we can consider following situation in the schema:

Date	Client	Office	Floor	Building	City	Representative
04/10/26	Client 1	2	10	ABC Tower	City 1	R1
04/11/26	Client 2	3	8	ABC Tower	City 1	R2
04/03/26	Client 3	4	5	QS Plaza	City 2	R8
04/04/26	Client 4	7	15	QS Plaza	City 2	R5
04/15/26	Client 5	10	18	Tech Heights	City 3	R10

- From the above schema, we can observe that the city location of each building is functionally dependent.
- ABC Tower -> City 1 | QS Plaza -> City 2 | Tech Heights -> City 3
  The city names are repeated. Hence, they are redundant because they store the same piece of information.
- Redundancy in schema wastes space as well as creating anomalies while performing modification, deletion, and insertion.
- Possible consequences:
  - If ABC Tower is in city 'City 5', and not in 'City 1', then we will have to update every single row where ABC Tower is mentioned. And, while updating, if any row(s) gets missed by the database, the whole database will become inconsistent.

- If 'Client 3' and 'Client 4' both cancel their meetings and we need to delete their records from the table, we will end up with no information on location details of 'QS Plaza'.

b. **Remove any existing partial dependencies and convert the logical schema to the Second Normal Form. Please remember that when performing schema decomposition you need to denote primary key for every new table as well as the foreign key that will allow us to reconstruct the original data.**

→ After removing the functional dependency in schema: **Office → Floor, Building, City** and **Client → Representative,** and converting the schema to 2NF, the tables are needed to decompose.

→ Putting all attributes of each partial dependency in a separate table, and making the part of primary key (of original table) as the primary key of new table, we get:
Office_Details(Office, Floor, Building, City)
Client_Details(Client, Representative)

To maintain a link to the original table:
Meeting_Details(Date, Client, Office)

The schema in 2NF is:
Office_Details(Office, Floor, Building, City)
Client_Details(Client, Representative)
Meeting_Details(Date, Client, Office); [Client references Client_Details(Client) and Office references Office_Details(Office)]

The above decomposition also maintains all the functional dependencies stated above in the question.
Note: I have marked the foreign key of the table in the 'red color' font, and the primary keys are underlined as per the standard.

c. **Remove any existing transitive dependencies to create a set of logical schemas in Third Normal Form. Again, remember to denote primary keys and foreign keys (including which primary key those foreign keys point to).**

→ To make the schema in 3NF, the transitive dependency of Office -> Building -> City.

→ The tables are decomposed into:
Office_Details(Office, Floor, Building); [Building references Building_Details(Building)]
Building_Details(Building, City)
Client_Details(Client, Representative)
Meeting_Details(Date, Client, Office); [Client references Client_Details(Client) and Office references Office_Details(Office)]

Part 3

Consider a table that stores information about students, student name, GPA, honors list and the credits that the student had completed so far.

(<u>First</u>, <u>Last</u>, GPA, Honor, Credits)

You are given the following functional dependencies

First, Last → GPA, Honor, Credits
GPA → Honors

a. Is this schema in Second Normal Form? If not, please state which FDs violate 2NF and decompose the schema accordingly.

→ The given schema is in 2NF.

→ There are no partial dependencies present (where a non-key attribute depends on only a part of a composite primary key).

a. Is this schema in Third Normal Form? If not, please state which FDs violate 3NF and decompose the schema accordingly.

→ The given schemas are not in 3NF.

→ There is a transitive dependency present, which violates the 3NF.

→ The transitive dependency which violates 3NF is:
First, Last -> GPA -> Honors

→ To make this in 3NF, the tables must be decomposed as:
(<u>First</u>, <u>Last</u>, GPA, Credits); [GPA references GPA of the other table]
(<u>GPA</u>, Honors)